

ZF-Microglia-AI — Complete User Guide

For zebrafish confocal microscopy — step by step, from zero to microglia labels.

Table of Contents

1. [What this plugin does](#)
 2. [Installation](#)
 3. [Getting the model files](#)
 4. [Opening the plugin in napari](#)
 5. [Tab 1 — Skin Remover](#)
 6. [Tab 2 — Create Labels](#)
 - [6a. Which tool is active — Pixel Classifier or Cellpose-SAM?](#)
 - [6b. Pixel Classifier — Union-Find Labels](#)
 - [6c. Cellpose-SAM Segmentation](#)
 7. [Tab 3 — Statistics](#)
 - [7a. Analysing cells by brain region \(optic tectum / hindbrain\)](#)
 - [7b. Intensity statistics per label](#)
 8. [Tab 4 — AI Tools](#)
 - [Email notification \(optional\)](#)
 - [8a. GT Annotation](#)
 - [8b. MONAI Training](#)
 - [8c. Cellpose-SAM Training](#)
 9. [Output files and folder structure](#)
 10. [Statistics CSV — all columns explained](#) — for the algorithm/formula behind each column instead, see the separate [STATISTICS GUIDE.md](#)
 11. [Setting up description backends](#)
 - [11a. Setting up email notification \(Gmail App Password\)](#)
 12. [Full workflow: from raw stack to labelled cells](#)
 - [Step 8a. Assign cells to optic tectum / hindbrain](#)
 13. [Reinstalling after an update](#)
 14. [Troubleshooting](#)
-

1. What this plugin does

You have a confocal microscopy stack of a zebrafish brain. The image contains the brain you care about plus skin, tissue, and background surrounding it.

This plugin does two things, in order:

Step A — Skin Removal (Tab 1): Uses a trained AI model (MONAI 3D U-Net) to automatically detect and remove everything outside the brain, producing a clean `brain_only` image where only the cells of interest remain visible.

Step B — Label (Tab 2): From the cleaned image, automatically finds and labels each individual cell as a separately numbered 3D region, using one of two methods — **Cellpose-SAM Segmentation**, a fine-tuned AI foundation model and the recommended choice, or the **Pixel Classifier**, an older-technology, threshold-based fallback for machines with no GPU. The tab shows whichever one matches your Tab 1 output automatically; see [Section 6a](#). Lets you sort, split, and edit labels before saving.

Step C — Analyse (Tab 3): Computes a comprehensive set of shape statistics for each labelled cell and exports them to a CSV file, with an optional AI-generated plain-language description per cell.

Every numeric field with a slider next to it (in all three tabs) is directly editable — click the number box and type an exact value instead of dragging the slider. Both stay in sync.

2. Installation

You need Python with napari already installed. Open a terminal and run:

```
pip install git+https://github.com/CTichy/ZF-Microglia-AI.git
```

All dependencies (PyTorch, MONAI, scikit-image, etc.) are installed automatically.

Mac with Apple Silicon (M1/M2/M3): The plugin automatically uses your GPU via Metal (MPS). No extra steps needed.

Windows / Linux with NVIDIA GPU: CUDA is detected and used automatically for Tab 1 inference and Tab 2 GPU-accelerated labelling (cupy-cuda12x, which has wheels for both platforms). Tab 3's *fastest* statistics path additionally uses cucim for GPU batch regionprops — this only has Linux wheels (RAPIDS/cuCIM has no native Windows build), so on Windows Tab 3 automatically falls back to a CPU-threaded path instead; nothing breaks, statistics just run somewhat slower. See Section 14 for details.

No GPU: Works on CPU too, just slower (~30–60 minutes per stack for inference).

3. Getting the model files

The plugin needs up to **two** trained checkpoints, depending on which labelling method you plan to use. Neither is bundled in the plugin.

Suggested layout — not required (the plugin remembers whatever path you browse to), but a tidy default if you'd rather not decide where to put things:

```
Documents/  
└─ zf-microglia-ai-models/  
    └─ MONAI/  
        └─ best_model_fullstack.pth  
    └─ Cellpose/  
        └─ <your checkpoint>
```

MONAI skin-removal model (required — Tab 1)

The AI model (~220 MB) that powers skin removal.

1. Download it:

<https://cloud.technikum-wien.at/s/kYQ4qq3Jsn4xEyY>

2. Save the file `best_model_fullstack.pth` into MONAI/ (or anywhere else easy to find).

3. Open the plugin, go to **Tab 1**, click the model [...] Browse button, and select the file — see “Model (.pth) — Browse button” under Section 5 below for the exact steps.

The plugin remembers the path — you only need to do this once per installation.

Cellpose-SAM checkpoint (optional — Tab 2, only if using Cellpose-SAM Segmentation)

Only needed if you plan to label cells with **Cellpose-SAM Segmentation** rather than the **Pixel Classifier** (see [Section 6a](#)). This is a project-specific fine-tuned Cellpose-SAM model (~580 MB), branch-weighted 3-fish checkpoint (multi3_bw, epoch 150).

1. Download it:

<https://cloud.technikum-wien.at/s/eFBJepk9DakDxyb>

2. Save the file `cpsam_microglia_512_multi3_bw_epoch_0150` into `Cellpose/` (or anywhere else easy to find).
3. Open the plugin, go to **Tab 2's** Cellpose-SAM Segmentation section, click the model **Browse** [...] button, and select the file. The path is remembered the same way as the MONAI model path.

If you don't have a checkpoint yet, use the **Pixel Classifier** instead — it needs no additional model file.

4. Opening the plugin in napari

1. Open a terminal and type `napari` to launch it.
 2. In the napari menu bar, click **Plugins**.
 3. Click **Main Panel (ZF-Microglia-AI)**.
 4. A panel appears on the right side with tabs: **Skin Remover**, **Create Labels**, **Statistics** (only appears once at least one Labels layer exists in the viewer), and **AI Tools** (always available — shows a disclaimer instead of hiding the tab if your GPU is missing or under the recommended 8GB, see [Section 8](#)).
-

5. Tab 1 — Skin Remover

Open TIF / IMS file

Click this button to open your confocal stack (.tif, .tiff, or .ims format).

- All channels in the file are loaded as separate napari layers, each coloured differently:
 - Channel 0 → gray
 - Channel 1 → green
 - Channel 2 → magenta
 - Channel 3 → cyan
- Voxel size (physical scale in μm) is read automatically from the file metadata and applied to all layers.
- The folder and filename are remembered for automatic output file naming (see Section 9).

Important: After loading, **click on the channel you want to process** in the Layers panel on the left. The plugin always runs on whichever image layer is currently selected (highlighted). For microglia, this is usually the green channel (ch1).

Model (.pth) — Browse button [...]

Shows the path to the AI model file. If it says “— no model selected —”:

1. Click the [...] button.
2. Navigate to where you saved the model file.
3. Select best_model_fullstack.pth and click Open.

The path is saved automatically to ~/.config/napari-zf-microglia-ai/config.json. Next time you open the plugin, the model is already loaded.

Input info display

Below the model path, the plugin shows:

- The name and shape of the currently selected layer
(e.g. "NT54_ch1" (300, 1024, 1024) uint16)
- The voxel dimensions: Z=1.0000 Y=0.1740 X=0.1740 μm

- The anisotropy ratio and the source of the scale information (from file metadata, from the layer scale, or default 1,1,1)

This is read-only — it updates automatically when you click a different layer.

MONAI Threshold

Range: 0.01 to 0.99 — **Default:** 0.30

The AI model outputs a probability map (0 = definitely not brain, 1 = definitely brain). This slider sets the cutoff: voxels above the threshold are classified as brain.

Value	Effect
0.20	More generous — includes uncertain areas; may keep some skin
0.25	Recommended — validated best results (Nathalie)
0.30	Previously documented default — superseded by 0.25
0.50	Stricter — may cut into brain edges

Post-processing (largest connected component + hole filling) cleans up most artefacts regardless of threshold. Keep it at 0.25 unless results look obviously wrong.

Erosion (vox)

Range: 0 to 15 voxels — **Default:** 0

After the brain mask is computed, this many voxels are stripped inward from the mask edge before applying it to brain_only. This removes a thin skin rim.

- **0:** No erosion — use the mask exactly as computed.
- **2–3:** Typical for zebrafish — removes a ~0.3–0.5 μm rim in XY or 2–3 μm in Z.

The brain_mask.tif saved to disk is **always the un-eroded mask**. Erosion only affects brain_only.tif.

Background (brain mode)

Four radio buttons controlling how background signal is handled after inference.

The background level is estimated automatically using the **mode** (most common intensity) of pixels inside the brain, computed from the result of inference. The mode represents the baseline scanner noise because background pixels vastly outnumber bright cell pixels.

Off

No background processing. $\text{brain_only} = \text{original volume} \times \text{brain mask}$. Everything outside the brain is zeroed; everything inside is the original signal unchanged.

1 — Remove background outside brain (inference)

Removes background-level pixels only in the region **outside** the brain boundary. The brain interior is fully protected — nothing inside changes. Useful for cleaning up outer tissue while leaving the brain completely untouched.

- Requires BG Threshold (see below).

2 — Remove background globally (full stack) ★ Recommended before labelling

Removes all pixels across the **entire stack** (including inside the brain) whose intensity falls at or below the background threshold.

Result: Only the actual signal (bright microglia, stained cells) survives. Background becomes zero everywhere, leaving clean isolated blobs with empty space between them — exactly what the Create Labels algorithm needs.

Use this option before creating labels.

The saved filename gets the suffix `_NoBG` (e.g. `NT54_ch1_brain_only_NoBG.tif`).

3 — Fill removed with random background

After skin removal, the region outside the brain is filled with **random noise** sampled from the actual background pixel distribution. The result looks like the original stack but with skin replaced by natural scanner noise — no hard black boundary at the brain edge.

- Uses Gaussian-filtered corner pixels as the noise pool ($\pm 2\sigma$ outlier removal) so the noise matches the real scanner texture.
- BG Threshold is not used in this mode.

The saved filename gets the suffix `_RndFill` (e.g. `NT54_ch1_brain_only_RndFill.tif`).

BG Threshold

Range: 0.00 to 2.00 — **Default:** 0.50

(Only active for background modes 1 and 2)

Fine-tunes the background removal threshold:

```
threshold = background_mode_value + BG_Threshold_offset
pixels ≤ threshold → removed (treated as background)
pixels > threshold → kept (treated as signal)
```

Value	Effect
0.00	Threshold = exactly the mode — removes only confirmed background
0.50	Previously documented default
0.60	Previously documented “recommended for microglia” — superseded by 1.40
1.40	Recommended for microglia labelling — validated best results (Nathalie)
2.00 (max)	Aggressive — may remove dim signal from thin cell protrusions

For microglia labelling, **1.40** typically produces the cleanest isolated blobs with good gaps between cells. If microglia are losing thin protrusions, lower the value.

Save checkboxes

- **Save brain_only.tif** (checked by default) — saves the brain-only volume with background removed
- **Save brain_mask.tif** (checked by default) — saves the binary mask as 0/255 uint8

Both files are saved in the output folder (see Section 9). The brain_only filename includes a background-mode suffix:

Mode	Suffix	Example filename
Off	(none)	NT54_ch1_brain_only.tif
1 — Exterior Removed	_ExtRm	NT54_ch1_brain_only_ExtRm.tif
2 — No Background	_NoBG	NT54_ch1_brain_only_NoBG.tif
3 — Random Fill	_RndFill	NT54_ch1_brain_only_RndFill.tif

Run Skin-Remover

Click to start processing. The button is greyed out while running; the status bar shows progress.

When complete, two new layers appear in napari:

- *_brain_mask — binary mask in cyan, semi-transparent
- *_brain_only[suffix] — the cleaned volume

Processing time: - NVIDIA GPU: ~30 seconds - Apple Silicon (MPS): ~5–10 minutes - CPU only: ~30–60 minutes

Verify MONAI Threshold / Erosion (GT Sweep)

The third of this plugin's three GT-sweep tools (the other two: Tab 2's BG Threshold/Erosion sweep for microglia labels, Tab 4's Cellpose-SAM epoch sweep). This one checks MONAI's own brain segmentation — is the current **MONAI Threshold** and **Erosion** combination actually the one closest to a hand-corrected brain mask?

Unlike the other two sweepers, this scores a single whole-volume mask, not multiple per-cell labels — Dice/IoU/precision/recall between the predicted brain mask and a GT mask directly, no complex-cell selection or bounding-box cropping needed.

1. **Image** — the raw volume to run inference on.
2. **GT brain mask** — a *hand-corrected* brain_mask.tif, e.g. from **GT Annotation** in Tab 4 (the polygon annotation tool's own rasterized output) — not a MONAI prediction.
3. **Threshold min/max/step** and **Erosion min/max/step** — define the grid. Defaults (0.15–0.35 step 0.05, 0–4 step 1) span 5×5=25 points centered on the recommended threshold.
4. Click **Run Threshold/Erosion Sweep**.

This is the cheapest of the three sweepers: MONAI's sliding-window inference (the only genuinely expensive, GPU-bound step) runs **exactly once**, producing a raw probability map. Every threshold and erosion value in the grid is then just a cheap re-threshold + largest-component/fill-holes + optional erosion on that same probability map — no reloading the model, no repeat sliding-window passes. A full 25-point grid typically finishes in well under a minute on GPU, and still works (just slower) on CPU/MPS since it uses the same device selection as **Run Skin-Remover**.

The report is a 2D grid (rows = Erosion, columns = Threshold, cells = Dice%), with your current Tab 1 slider values marked. Same policy as the other sweepers: once the sweep finishes, its best point is **applied directly to the Tab 1 Threshold/Erosion sliders and saved to config** — no manual copy-over needed, and the recalibrated values persist across napari restarts. The report still shows the full grid so you can see how confident the recommendation is.

6. Tab 2 — Create Labels

Before using this tab, run Tab 1 first, then click the resulting brain_only layer in the Layers panel to select it. Which section of Tab 2 appears depends on which background mode you used — see 6a below.

6a. Which tool is active — Pixel Classifier or Cellpose-SAM?

Use **Cellpose-SAM Segmentation (6c)** if you have a GPU. It's a fine-tuned AI foundation model and handles branching, overlapping, and faint microglia far better than classical thresholding — it's the labelling method every real result in this project has actually used. The **Pixel Classifier (6b)** is an older, simpler threshold-and-stitch tool kept around as an initial aid for machines with no GPU at all; treat it as a fallback, not a first choice, when a GPU is available.

Tab 2 shows **exactly one** of the two labelling methods below, chosen automatically from the active layer's filename suffix. Select a different layer in the Layers panel and Tab 2 switches live — no manual toggle needed.

Active layer ends in	Section shown	Produced by Tab 1 option
_ExtRm	Cellpose-SAM Segmentation (6c)	Option 1 — Remove background outside brain
_NoBG	Pixel Classifier (6b)	Option 2 — Remove background globally
_RndFill	<i>Neither</i> — this output is for presentation/visualisation only	Option 3 — Fill removed with random background
anything else (e.g. the raw channel)	<i>Neither</i> , with a hint on what to select	—

So the choice is really made back in **Tab 1, Step 5**: pick **Option 1** if you plan to segment with Cellpose-SAM, or **Option 2** if you plan to use the Pixel Classifier.

The **Sort by / Resort Labels**, **Split Label**, and **Save Labels** tools (Section 6, further down) only appear once one of the two sections above is showing — with no _ExtRm/_NoBG layer selected, there's nothing yet to sort, split, or save. Likewise, **Tab 3 — Statistics** only appears once at least one Labels layer exists in the viewer.

6b. Pixel Classifier — Union-Find Labels

Fully self-contained: Gaussian smooth → threshold → per-slice 2D connected components → overlap-based union-find into 3D objects → volume filter → sequential renumber. Shown when the active layer ends in _NoBG (background removed everywhere, not just outside the brain) — needs no additional model file.

Smooth σ XY

Range: 0.0 to 5.0 — **Default:** 1.0 — **Recommended:** 1.5

Controls the softness of blob contours **within each 2D slice** (the XY plane).

Gaussian smoothing is applied before thresholding each slice. This rounds jagged pixel edges and fills tiny holes within the same cross-section.

Value	Effect
0.0	No smoothing — raw pixel edges
1.5	Recommended — solid, rounded blobs with preserved shape
3.0+	Heavy — risk of merging nearby cells within the same slice

Do not confuse with Smooth σ Z. They serve completely different purposes.

Smooth σ Z

Range: 0.0 to 5.0 — **Default:** 0.5 — **Recommended:** 3.0

Controls **cross-slice connectivity** — how easily the algorithm links blobs in neighbouring Z slices into a single 3D object.

A microglia that disappears for 1–2 slices (due to low signal or a thin neck) and reappears will be correctly merged into one 3D object when σ Z is high enough.

Why σ Z = 3.0 while σ XY = 1.5?

Zebrafish confocal stacks are highly anisotropic: each Z slice is $\sim 1\ \mu\text{m}$ thick while each XY pixel is $\sim 0.17\ \mu\text{m}$. So $\sigma Z = 3.0$ spans $\sim 3\ \mu\text{m}$ physically, while $\sigma XY = 1.5$ spans only $\sim 0.26\ \mu\text{m}$.

A microglia is typically $10\text{--}20\ \mu\text{m}$ in diameter. Two microglia need to be closer than $\sim 3\ \mu\text{m}$ in Z for $\sigma Z = 3.0$ to risk merging them — which is uncommon in practice. This has been validated safe for zebrafish 4dpf microglia.

Value	Effect
0.0	No cross-slice smoothing — each slice fully independent
0.5	Minimal — only adjacent slices with strong overlap connected
3.0	Recommended for zebrafish — bridges 1–3 slice gaps
5.0+	Very aggressive — may link cells at different Z depths

Min overlap (%)

Range: 1 to 100 — **Default:** 10%

Two blobs in adjacent slices are recognised as the **same 3D cell** only if they share at least this fraction of the smaller blob's area:

```
overlap_ratio = shared_pixel_count / area_of_smaller_blob
if overlap_ratio ≥ min_overlap% → same object (linked)
```

- **Lower (5%):** Permissive — small touching fragments are linked.
- **Higher (30%):** Strict — only well-aligned blobs linked; isolated particles stay separate.
- **Start at 10%** and increase if too many fragments are joined, or decrease if cells are being cut across slices.

Min volume (vox)

Range: 5000 to 10000 — **Default:** 7500

After all 2D blobs are linked into 3D objects, any object smaller than this voxel count is deleted as noise.

Value	When to use
5000	Keep smaller objects — may include noise

Value	When to use
7500	Default — validated for adult zebrafish microglia
10000	Keep only large objects — use if many small debris remain

Zebrafish microglia at 4dpf typically occupy 15,000–50,000 voxels at standard resolution.

Create Labels

Click to run the 3D labelling algorithm. Processing runs in a background thread — the button is disabled until complete.

When done, a *_labels layer appears in napari with each detected cell shown in a different colour. The console prints how many labels were found.

Verify BG Threshold / Erosion (GT Sweep)

Answers the same kind of question as Tab 4’s “Verify Best Epoch” tool, but for the Pixel Classifier path instead of Cellpose-SAM: is the current **BG Threshold** (Tab 1) and **Erosion** (Tab 1) combination actually the one that produces microglia labels closest to ground truth, or does a nearby combination do better?

1. **GT image** — the full-fish raw/brain_only image, same one Tab 1 ran on.
2. **brain_mask.tif** — the *raw* (un-eroded) mask Tab 1 saves. MONAI inference itself is **not** re-run by this sweep — it only varies what happens *after* inference (erosion, background thresholding, labelling), so it needs an already-computed mask from a normal Tab 1 run rather than the model checkpoint.
3. **GT labels** — the corrected ground-truth microglia label volume for that fish.
4. **BG Threshold min/max/step** and **Erosion min/max/step** — define the grid. Defaults (1.0–1.8 step 0.2, 0–4 step 1) span 5×5=25 points centered loosely on the recommended BG Threshold.
5. Click **Run BG/Erosion Sweep**. For each grid point, it: finds the N most complex GT cells (same branch-count ranking as the Cellpose-SAM sweep, computed once), applies that erosion + BG Threshold to each cell’s cropped region, runs Create Labels (using this section’s

own σ_{XY} / σ_Z / Min volume above) on the crop, and best-IoU-matches the result against GT.

This sweep is considerably cheaper than the Cellpose-SAM one: MONAI inference only ever runs once (outside this tool, via a normal Tab 1 run), and neither Erosion nor BG Threshold require reloading a model — Erosion is a cheap morphological operation and the background “mode” statistic is computed once per erosion value, not once per grid point. A full 25-point grid typically finishes in minutes. It also runs on CPU if no GPU is present (Create Labels already has a CPU fallback), unlike the Cellpose-SAM sweep.

The report is a 2D grid (rows = Erosion, columns = BG Threshold, cells = average IoU%), with your current Tab 1 slider values marked and compared against whatever the sweep found best. Once the sweep finishes, its best point is **applied directly to the Tab 1 BG Threshold/Erosion sliders and saved to config**, the same way the MONAI Threshold/Erosion sweep does — the report is still shown in full so you can judge how confident the recommendation is (only a handful of cells and grid points are tested, so a narrow win shouldn’t be trusted as blindly as a wide one).

This tool depends on Erosion and BG Threshold actually composing correctly in Tab 1’s own pipeline. An earlier version of `_on_run` silently discarded Erosion whenever any Background mode was active (the final mask always used the raw, un-eroded mask in that code path) — fixed as of this version, so Erosion now has a real, sweepable effect in the recommended Background mode 2 workflow this tool targets.

6c. Cellpose-SAM Segmentation

Shown when the active layer ends in `_ExtRm` (background removed only outside the brain — the interior is left intact for Cellpose-SAM to see). Runs `do_3D` Cellpose-SAM inference, then a 3-component-GMM cleanup pass, a Krendl safe-merge pass (rejoins sub-threshold fragments based on gap size and contact area), and a large-contact merge pass (catches cells accidentally split through a thick junction).

Requires a Cellpose-SAM checkpoint — see [Section 3](#). This is a project-specific fine-tuned model, not shipped with the plugin.

Model (.pt/checkpoint) — Browse button [. . .]

Browse to your trained Cellpose-SAM checkpoint file. The path is remembered across sessions, the same way the MONAI model path is remembered in Tab 1.

Cellprob threshold

Range: -6.0 to 6.0 — **Default:** -2.5

Cellpose-SAM's own confidence cutoff for what counts as foreground (cell) vs. background. Lower (more negative) values are more permissive — they recover more of a cell's thin, dim protrusions but can also let in more noise.

Flow threshold

Range: 0.0 to 1.0 — **Default:** 0.4

Rejects predicted objects whose internal flow field doesn't self-consistently point back to a single centre (a sign of a malformed or noisy detection). Lower values are stricter — fewer, more confident objects.

Safe-merge max gap (vox)

Range: 0 to 20 — **Default:** 2

During the Krendl safe-merge pass, two fragments separated by a gap up to this many voxels are considered for merging into one cell (in addition to the contact-area check below).

Safe-merge min contact (vox)

Range: 0 to 200 — **Default:** 10

Minimum shared-boundary voxel count required between two fragments before the safe-merge pass will join them. Higher values require a more substantial touching surface before merging.

Safe-merge GT-min volume (vox)

Range: 0 to 50000 — **Default:** 10230 (the historical “smallest real microglia volume seen in validated GT data”)

The volume, in voxels, below which the safe-merge pass treats a fragment as *not yet* a whole cell and a candidate to merge into something else. Rather than trust a single frozen historical number forever, the **Verify Cellprob / Large-contact (GT Sweep)** tool below measures this directly from whatever GT labels volume you sweep against (the true minimum labeled-cell volume in that GT) and recalibrates it automatically — you shouldn't normally need to set this by hand.

Large-contact merge (vox)

Range: 1 to 2000 — **Default:** 20

A second, separate merge pass for large blobs that got split apart through a thick junction (more contact area than the safe-merge pass alone would normally join). Raise this if large cells are still coming out fragmented; lower it if separate cells are being wrongly joined.

Run Cellpose-SAM Segmentation (button)

Click to start. `do_3D` inference is slow — it can take **hours** for a full-size fish — and runs in a background thread with live status updates, so napari itself stays responsive while it works. When complete, a `*_labels` layer appears, exactly as with the Pixel Classifier.

If this button errors with No module named 'cellpose', install it in your environment: `pip install cellpose` (already listed in `environment.yml/environment-mac.yml` for fresh installs — see Section 14).

Verify Cellprob / Large-contact (GT Sweep)

Sweeps **Cellprob** × **Large-contact merge** against a full-fish GT labels volume, scored with the exact same whole-fish Hungarian-matched methodology as **Score Against GT** in Tab 3 — this is how the current defaults were actually found historically (e.g. the `cellprob=-2.5/large_contact=20` combination), now automated instead of requiring a CLI sweep script.

1. **Image / GT labels** — a full-fish `brain_only` image + its corresponding GT labels volume.
2. **Voxel scale Z/XY** — drives the `do_3D` anisotropy parameter (Z/XY ratio); independent of whatever's open in the viewer.

3. **Cellprob min/max/step** and **Large-contact min/max/step** — define the grid.
4. Click **Run Cellprob/Large-contact Sweep**. Uses this section's current **Flow**, **Safe-merge max gap**, and **Safe-merge min contact** values above — only Cellprob and Large-contact vary.

Cellprob requires a real do_3D re-inference per value — it changes what Cellpose-SAM actually predicts, so this is the expensive, GPU-preferred dimension (same device fallback as Run Cellpose-SAM Segmentation). **Large-contact is cheap** — it's a post-processing merge threshold applied after do_3D + GMM cleanup + Krendl safe-merge, so the sweep runs do_3D (+ GMM + safe-merge) exactly once per Cellprob value, then varies Large-contact freely on that same intermediate result — the same shortcut this project's own research scripts have long used (`--skip_inference`).

Because of that, total sweep time scales with the number of **Cellprob** values only, not the full grid size — a 5×5 grid costs about the same as 5 individual do_3D runs, not 25. **Stop Sweep** cancels between Cellprob values (not mid-inference or mid-Large-contact). The report is a 2D grid of Score ($TP - 0.5 \times (FP + FN)$), with your current Tab 2 slider values marked.

Safe-merge GT-min volume is also recalibrated every time you run this sweep — measured directly from the GT labels volume's own smallest labeled cell, rather than a frozen historical constant (see the field's description above). Once the sweep finishes, its best Cellprob/Large-contact point **and** the measured GT-min are all applied directly to the Tab 2 sliders and saved to config — no manual copy-over needed.

Build GT-Correction Package

Packages a Krendl segmentation result for external manual correction — the exact file layout this project has assembled by hand for every fish sent out for ground-truth creation. The corrected result becomes future training/GT data, closing the loop between inference and the training tools in Tab 4.

1. **Fish stem** — the identifier used to name every file in the package (e.g. NT39-3dpf-D1F4_2024-09-05_15.38.01).
2. **Source image** — the brain_only image the segmentation ran on.

3. **Krendl masks** — the output of **Run Cellpose-SAM Segmentation** above. Becomes `<stem>_masks_corrected.tif` — the file the reviewer edits first (“start here” per the guide).
4. **Raw Cellpose masks** (optional) — the pre-merge `do_3D` output, if you have it, included as `<stem>_cp_masks_3D.tif` for reference only (not corrected).
5. **Creation guide** (optional override) — defaults to this project’s own `GROUND_TRUTH_CREATION_GUIDE.md`; only set this if it lives somewhere else on your machine.
6. **Output folder** — where the package folder and `.zip` are created.

Click **Build GT-Correction Package**. Output:

```
<output folder>/
├── <stem>_GT_package/
│   ├── GROUND_TRUTH_CREATION_GUIDE.md
│   ├── <stem>_masks_corrected.tif
│   ├── <stem>_cp_masks_3D.tif          (only if provided)
│   ├── <stem>_cell_statistics.csv      (label/volume/centroid/bbox
– quick reference, not the full Tab 3 output)
│   └── <stem>_brain_only_ExtRm.tif
└── <stem>_GT_package.zip              (the folder above, zipped)
```

The statistics CSV is deliberately minimal (label, volume, centroid, bounding box) — a quick reference for someone correcting labels, not the full ~51-column Tab 3 Statistics output.

Sort by / Reverse order / Resort Labels

After creating (or loading) labels, you can renumber them by a criterion of your choice.

Sort by dropdown:

Option	Meaning	Default order
Size	Number of voxels	Largest = label 1
Centroid Z	Z coordinate of centre	Smallest Z = label 1
Centroid Y	Y coordinate of centre	Smallest Y = label 1
Centroid X	X coordinate of centre	Smallest X = label 1

Reverse order checkbox — inverts the ordering (e.g. smallest = label 1 for Size).

Click **Resort Labels** to apply. The active Labels layer is renumbered 1...N in the chosen order, in place. This is useful for consistent numbering across samples or for matching cells to a reference atlas.

Split Label

Splits a single merged label (a blob where two or more cells are stuck together) into separate parts using a 3D watershed algorithm.

The watershed approach finds the **thinnest neck** connecting two large volumes and cuts there — it does not use a simple distance threshold or Euclidean splitting.

Target label

The label number of the blob you want to split. You can type it directly, or:

1. In the napari viewer, hover over the blob and read the label number shown in the status bar.
2. Click the blob to select it in the Labels layer.
3. Click **Use selected** — the label number is filled in automatically.

Use selected

Reads the currently selected label from the active napari Labels layer and fills it into the Target label spinner. Click the blob in napari first, then click this button.

Split into N parts

Range: 2 to 10 — **Default:** 2

How many separate pieces the blob should be divided into. The algorithm searches for the N largest sub-volumes (separated at their thinnest necks) and cuts between them.

If the blob genuinely has only one major volume (no neck), splitting may fail or produce uneven results. Increase Smooth σ or use a lower Min distance if that happens.

Smooth σ (Split)

Range: 0.0 to 3.0 — **Default:** 1.0

Gaussian smoothing applied to the distance transform before searching for peaks. Higher values smooth out the distance map, making the algorithm more robust to surface noise but less sensitive to subtle necks.

- **0.5–1.0:** Suitable for most cases.
- **1.5–2.0:** Use if the split point jumps around — smoother distance field = more stable result.
- **0.0:** No smoothing — very sensitive to surface texture.

Min distance

Range: 1 to 30 voxels — **Default:** 5

Minimum voxel distance required between accepted seed peaks. If two candidate peaks are closer than this, only the stronger one is kept.

- **Too high:** The two centres of a closely-packed double-blob may be rejected as “too close” → fewer than N peaks found → error.
- **Too low:** Surface noise peaks may be accepted as separate centres → wrong split point.
- **5 voxels** works well for microglia-sized cells.

Split Label (button)

Click to run. The original blob is replaced in-place:

- The original label number is kept for the **first** part (the largest sub-volume).
- New label numbers (`max_existing + 1`, `max_existing + 2`, ...) are assigned to the remaining parts.

The cut is **interface-only**: exactly the voxels at the boundary between parts are removed, creating a 1-voxel gap. The outer surface of each part is not touched — thin protrusions are preserved.

If the algorithm cannot find N distinct sub-volumes, an error message is shown. Try reducing Smooth σ or Min distance.

Save Labels

Opens a file-save dialog pre-filled with the output folder (see Section 9) and the current layer name as the filename. Choose a location and filename, then click Save.

Labels are saved as int32 TIFF. Each voxel value = label number (0 = background).

Save Labels is separate from Create Labels by design. This lets you edit labels in napari (split, delete, merge) before saving the final result.

After saving labels, switch to **Tab 3 — Statistics** to compute measurements for each cell.

7. Tab 3 — Statistics

This tab computes a comprehensive set of morphological, spatial, intensity, and brain-region measurements for every label and saves them to a CSV file. It is intentionally separate from Tab 2 so there is room to configure all options comfortably before clicking Generate.

For what each output column means, see [Section 10](#) below. For the algorithm/formula/library behind each one — useful if you’re auditing a result or citing the method — see the separate [STATISTICS GUIDE.md](#).

Make sure a Labels layer is selected in napari before using this tab.

Description backend

Selects the engine used to generate the plain-language description column in the CSV.

Option	Internet	Cost	Notes
Rule-based (offline)	No	Free	Always available; template-based sentences
	No	Free	

Option	Internet	Cost	Notes
Ollama (local, free)			Runs a local LLM on your machine
OpenAI API (paid)	Yes	Pay-per-token	GPT-4o-mini recommended for low cost
Claude API (paid)	Yes	Pay-per-token	claude-haiku-4-5 recommended for low cost

See Section 11 for detailed setup instructions for each backend.

Ollama sub-panel (shown when Ollama is selected)

- **Endpoint:** URL where Ollama is running. Default: `http://localhost:11434`. Change this if Ollama runs on a different machine or port.
- **Model:** The Ollama model name to use (e.g. `llama3`, `mistral`, `phi3`). Must be pulled first (`ollama pull llama3`).

API sub-panel (shown for OpenAI or Claude)

- **API Key:** Your secret API key. Shown as dots (password field). **Not saved to disk** — you must re-enter it each session.
- **Model:** The model identifier (e.g. `gpt-4o-mini` for OpenAI, `claude-haiku-4-5-20251001` for Claude).
- **Base URL:** Optional. Leave blank unless you use an OpenAI-compatible proxy or self-hosted endpoint.

Intensity statistics (optional)

Image layer dropdown — select an Image layer from your napari session (or leave as “None” to skip).

When an image layer is selected, three additional columns are computed per label using the raw intensity values inside each cell’s mask:

- `mean_intensity` — average pixel intensity inside the label

- `integrated_intensity` — total sum of all pixel values (proportional to total fluorescent material)
- `intensity_cv` — coefficient of variation (std / mean) — a measure of how uniform the signal is; 0 = perfectly uniform, high values = heterogeneous staining

Select the microglia channel (usually the green channel, ch1) for biologically meaningful results.

Brain regions (optional)

Assigns each cell to a named anatomical brain region and computes its distance to the nearest region boundary.

Boundary lines dropdown — select a Shapes layer containing one or more line shapes, or leave as “None” to skip.

Region names text field — enter the region names separated by commas, listed anterior to posterior. For N boundary lines, provide exactly N+1 names.

Example: If you draw one line separating the optic tectum from the hindbrain, enter:

Optic tectum, Hindbrain

Fish orientation and axis convention:

In these stacks, the fish lies along the **X axis** with the head pointing toward X = 0 (anterior = small X, posterior = large X). Y runs from 0 to 2048 top-to-bottom. The optic tectum / hindbrain boundary therefore runs roughly **top to bottom along Y**, separating the left part of the image (optic tectum, small X) from the right part (hindbrain, large X).

How to draw region boundaries:

1. In the napari toolbar, click **New shapes layer** (or add via Layers → Add shapes layer).
2. Select the **path** tool in the toolbar (a polyline — click once per vertex, double-click to finish). Use **path** rather than **line** so you can follow the curved anatomy of the optic tectum / hindbrain boundary.
3. Click along the boundary curve **from top to bottom** (Y = 0 toward Y = max), following the anatomical contour. The optic tectum will

- be on the left of your drawn path (smaller X), the hindbrain on the right (larger X). Double-click on the last point to finish.
4. For multiple regions, draw one path per boundary, each running top to bottom.
 5. Select the Shapes layer in the **Boundary lines** dropdown and type your region names.

The boundaries are sorted automatically by mean X position of their vertices (leftmost = most anterior). For each cell, the plugin finds the nearest segment on the boundary curve and uses its orientation to determine which side the cell falls on (left = more anterior region, right = more posterior region).

Two additional columns are added to the CSV:

- `brain_region` — name of the region this cell belongs to
 - `region_boundary_dist_um` — distance in μm to the nearest region boundary line
-

Generate Statistics (button)

Click to compute. Runs in a background thread. When complete:

- A CSV file is saved to the output folder (see Section 9), named `{source_file_stem}_statistics.csv`.
- The status line shows how many labels were processed.

The CSV contains one row per label with up to 45 columns depending on which optional features are enabled. See Section 10 for a full description of every column.

Score Against GT

Whole-fish instance-segmentation scoring: Hungarian-matched TP/FP/FN/Score plus mean IoU/Dice (over matched pairs only), between any two Labels layers already loaded in the viewer. This is the same methodology (`compare_pred_gt.py`) this project has used to validate essentially every real modeling decision — checkpoint picks, cellprob/large_contact tuning, before/after model comparisons — ported into the plugin instead of staying a CLI-only script.

1. **Predicted labels / GT labels** — pick any two Labels layers from the dropdowns (same shape required).

2. **IoU threshold for a match** — the minimum IoU for a predicted object to count as a true positive for a given GT object (default 0.5, matching this project’s historical convention).
3. Click **Score Against GT**.

Runs synchronously (pure CPU, `scipy.optimize.linear_sum_assignment`) — fast enough at typical whole-fish object counts (tens to low hundreds) that a background thread isn’t needed. Object pairs are only compared when their bounding boxes actually intersect (IoU is necessarily 0 otherwise), so this stays fast without ever missing a possible match.

Score = **TP** – **0.5×(FP + FN)**, matching the exact formula used throughout this project’s historical result tables. The report lists every matched pair (IoU%, Dice%, voxel counts, size delta), plus the FN (missed GT) and FP (spurious predicted) object IDs.

This is a genuinely different tool from the three GT-sweep tools elsewhere in the plugin (Tab 1/2/4): those each test a handful of parameter combinations against a handful of complex cells (or one mask) as a fast proxy; this scores one specific pair of label volumes completely, the way a final reported result would be scored.

8. Tab 4 — AI Tools

Always visible, regardless of GPU. A banner at the top reports your GPU situation and adjusts its tone accordingly — checked once when napari starts:

Situation	Banner
No CUDA GPU detected	Red, bold: training/ inference will run on CPU — can take days to months for a full run instead of hours. Still usable for small experiments.
CUDA GPU present, under 8GB VRAM	Amber, bold: training may still work with a reduced batch_size (try 2, or even 1) but could be slow or hit out- of-memory errors.
CUDA GPU present, ≥8GB VRAM	

Situation	Banner
	Green, quiet confirmation — no action needed.

This used to be a hard gate — the whole tab was hidden below 8GB VRAM. Changed deliberately: a smaller GPU, or none at all, doesn't mean the tools are useless, just slower, or in need of a smaller `batch_size`. GT Annotation itself has never needed a GPU either way.

Email notification (optional, shared by both groups)

A small panel at the top of the tab, above the MONAI/Cellpose-SAM switch, applying to whichever group you launch: **Notify email, SMTP server/port, SMTP username, SMTP password**. Leave **Notify email** blank to disable it entirely — that's the default.

When set, one email is sent whenever a training run stops — finishes normally, crashes, or gets early-stopped — with the best checkpoint's epoch/metric value and the exit status. Free with any Gmail account, no other signup needed. **See Section 11a for the full step-by-step setup** (turning on 2-Step Verification, generating a Google App Password, and what to type into each field) — the short version: server `smtp.gmail.com`, port 465, username = your Gmail address, password = a Google App Password, not your normal Gmail password, which won't work here.

Only the address/server/port/username are saved between napari sessions — the password never is (same policy as the API key in the Statistics tab), so you'll need to re-enter it before each launch. If **Notify email** is filled in but username/password is missing, Launch Training refuses to start until you either fill both in or clear the email field.

Why this works even if napari is closed the whole time: the notification isn't sent by the GUI's live polling (which only runs while napari is open) — instead, the launched background process itself is a small wrapper that runs the real training command, waits for it to finish, *then* sends the email, before exiting. That wrapper is what's detached and survives napari closing, exactly like the training script itself, so the email still arrives on schedule whether or not you ever reopen napari to see it. Clicking **Stop Training** kills the whole thing (wrapper included) before it reaches the email step, so a manual stop doesn't send one — only unattended completions/crashes do.

A switch below that picks one of two mutually-exclusive groups. Only one is shown at a time.

8a. GT Annotation

Hand-draw polygon boundaries on key slices to create brain/skin ground-truth masks — the manual annotation step that produces training data for the MONAI model.

1. **Image layer** — pick the Image layer to annotate from the dropdown (populated from whatever's open in the viewer — use **Open TIF / IMS file** in Tab 1 first if nothing is loaded yet). Selecting a layer here automatically creates a brain_polygons Shapes layer (yellow) if one doesn't already exist.
2. **Draw polygons** — select the brain_polygons layer in the Layers panel, choose napari's polygon tool, and trace the brain boundary on key slices — roughly every 10 slices (e.g. 0, 10, 20, 30...). You don't need to draw every slice: the polygon on slice 90 is automatically propagated to all slices beyond it.
3. **1. Interpolate Polygons** — smooths and resamples each drawn polygon to 96 points, then interpolates point-to-point between key slices along Z. Produces a brain_polygons_interpolated layer (cyan) — review it before continuing.
4. **2. Generate Masks** — rasterizes the interpolated polygons into a brain mask, saves brain_mask.tif, skin_mask.tif, original.tif, brain_only.tif, skin_only.tif, and both polygon .npz files to <source_folder>/<source_stem>/ (the same output-folder convention as every other tab — see Section 9). Also adds brain_mask, skin_mask, and a new brain_only layer to the viewer, without touching or hiding any of your other layers.

Note: unlike Tabs 1-3, this section doesn't have its own file-open button — it always annotates whichever file was most recently opened via **Open TIF / IMS file** in Tab 1, matching that same output-folder convention.

8b. MONAI Training

Prepares training data and launches MONAI U-Net training — the model Tab 1 uses for skin removal.

Prepare Training Data converts raw+GT fish folders (the output layout GT Annotation produces) into the HDF5 dataset the trainer needs. Leave the brain/skin directory fields blank to use the training script's own built-in defaults, or list your own comma-separated paths. Takes a few minutes; runs in the background without blocking the UI. On success, auto-fills the Train MONAI section's data directory.

Train MONAI U-Net launches the actual training run — configure epochs/batch_size/lr/val_every/ckpt_every/GPU index, optionally point resume at an existing checkpoint to continue training instead of starting fresh, then click **Launch Training**. See [How training launches work](#) below for the shared **Patience (checkpoints)** early-stopping field and what happens next.

8c. Cellpose-SAM Training

Extracts fine-tuning crops and launches Cellpose-SAM training — the model Tab 2's Cellpose-SAM Segmentation uses.

Extract Training Crops reads a `_statistics.csv` (from Tab 3) plus the matching brain_only image and labels TIFFs, and extracts single/double/triple/quadruple bounding-box crops per cell (the cell alone, plus its 1st/2nd/3rd nearest neighbours) — duplicate crops (same cell set) are saved only once. Optionally set `val_cells` (comma-separated cell IDs) for a cell-level train/val split. Takes seconds to a few minutes. On success, auto-fills the Train Cellpose-SAM section's data directory. **This is an older extraction method** — see below for the one actually used by every real training run in this project since May 2026.

Extract XXYZ Patches (current production method) generates 2D training crops in all three orientations from a full-fish image + GT labels pair: **XY** at native resolution, **XZ/YZ** with the Z axis stretched by the voxel-scale-derived anisotropy so all three orientations end up at the same effective pixel scale. Only crops containing GT signal are kept, up to `max/orientation` per orientation. This is the method behind every real Cellpose-SAM training dataset in this project (`train_cellpose_512`, `_multi`, `_multi3`).

- **crop_size / crops/slice / max/orientation / min_gt_pixels / seed** — control crop dimensions and how many are sampled; defaults (512 / 5 / 320 / 10 / 42) match what's been used for every real dataset so far.
- **Clean truncated labels after generation** (checked by default) — a crop framed around one target cell can also graze the corner of a

different nearby cell purely by chance, sometimes showing only a tiny sliver of it. Left in, that sliver is still a valid-looking label with a wildly wrong flow-center target (Cellpose points flow vectors toward each object's own centroid — a fragment's visible centroid is nowhere near the real cell's center). With this on, any label whose crop-visible pixel count falls below **Minimum visible fraction to keep a label** (default 90%) of its true full-slice cross-section gets zeroed out of that crop — automatically, right after generation, not as a separate step you have to remember. A crop's own intended target cell is essentially never affected (it's already well above this threshold in the crop it was generated for); this specifically catches incidental neighbors. Before writing anything, the whole crop folder is backed up to <folder>_pretrunc_backup — skipped on a second run if that backup already exists, so re-running never overwrites an earlier backup with already-cleaned files.

- This is the exact fix already applied by hand to this project's real training data (D1F1/D1F2/D1F4, 2026-08-05) — now the default behavior for every future extraction, not a one-off research script.

Train Cellpose-SAM launches fine-tuning — configure `n_epochs/` `batch_size/save_every/log_every/lr`, then click **Launch Training**. The pretrained field defaults to whatever checkpoint is already loaded in Tab 2's Cellpose-SAM Segmentation section — i.e. by default this **continues training from where Tab 2 left off**, though you can browse to a different starting checkpoint (or type a builtin name like `cpsam`) if you want to start fresh. `branch_weight/branch_radius` control the project's branch-weighted loss (weights thin/branch-tip pixels more heavily during training so the model doesn't under-segment fine processes) — set `branch_weight` to 0 to disable it and use the standard Cellpose loss instead.

Calibrate branch_radius (from GT) measures the real branch thickness of actual GT-labeled cells instead of guessing `branch_radius` by hand. Browse to a GT labels volume, set **scale Z/scale XY** to match its voxel scale, and click **Calibrate branch_radius**. The tool 3D-skeletonizes every labeled cell, decomposes each skeleton into branch segments, measures each segment's mean diameter via an anisotropic distance transform, and takes the **thinnest quartile** (the distal branch tips — the fine processes `branch_weight` exists to protect, as opposed to thick soma-adjacent segments) as the basis for the recommendation, converting that radius from microns to pixels at the given scale. The result is applied directly to the `branch_radius` field above and saved to

config — no manual copy-over. This can take anywhere from several seconds to a couple of minutes depending on how many cells are in the GT volume; it runs in a background thread so napari stays responsive.

Verify Best Epoch (GT Sweep) answers a specific question the recommended checkpoint alone can't: `test_loss` (what picks the recommendation) is a proxy for segmentation quality, not the real thing, and checkpoints often plateau within noise of each other. This tool checks the recommendation against actual ground truth on a small, deliberately hard sample:

1. **GT image / GT labels** — browse to a full-fish raw/brain_only image and its corrected ground-truth label volume (the same pair used to build training crops). Doesn't need to be a fish the current model was trained on, but usually is — it's the same file pair you'd use to sanity-check any checkpoint.
2. **Recommended epoch** — click **Read from pointer file** to pull it from `<model_name>_best_recommended.txt` (uses this section's own Data dir/model_name from Train Cellpose-SAM above), or type it in manually if no pointer exists yet.
3. Click **Run Epoch Sweep**. The tool:
 - Finds the **N most morphologically complex cells** in the GT volume (default 5) — ranked by skeleton branch count (most-branched first), sphericity as a tiebreak, *not* by cell size. A large cell is often simple/amoeboid; a genuinely branchy one is what actually stresses the branch-weighted loss.
 - Crops each to its bounding box + padding (default 15 vox Z, 40 vox XY).
 - Runs `do_3D` inference at the recommended epoch plus **N checkpoints below and above it** (default 2 and 2 — a 5-epoch × 5-cell = 25-inference sweep by default), best-IoU-matches each prediction against its GT cell, and averages.
 - Reports a table (same format as the project's earlier manual sweeps) plus a plain confirm/disagree verdict against the recommended epoch.

This can take a while — each `do_3D` call is a few minutes, so a default 5×5 sweep is roughly 30 minutes to a couple of hours depending on cell/crop size and GPU. **Stop Sweep** cancels between checkpoints (not mid-inference) and still shows whatever completed. Unlike Launch Training, this does **not** run as a detached process and does **not** survive closing napari — it's meant to be watched, in the same tier as Extract Training Crops, not the multi-hour/day training jobs.

If the sweep disagrees with the recommendation, it's applied automatically: the recommended-checkpoint pointer (`<model_name>_best_recommended.txt`) is rewritten to the sweep-confirmed epoch, and that checkpoint is loaded as Tab 2's active Cellpose-SAM model — the same effect as browsing to it by hand, but done for you. The report is still shown in full so you can judge how confident the disagreement actually is (only a handful of cells and epochs were tested).

How training launches work

Both “Launch Training” buttons (MONAI and Cellpose-SAM) start a **detached background process** rather than running inside napari itself — the command runs via `conda run -n <env> --no-capture-output <script> ...`, launched so it keeps running even if you close napari (technically: `setsid()` on Linux/Mac, `CREATE_NEW_PROCESS_GROUP` | `DETACHED_PROCESS` on Windows). This works identically on all three platforms without needing `tmux` (which isn't available on Windows at all).

- **Live status** — a log-tail view in the GUI refreshes every 8 seconds (deliberately coarse — there's no benefit to checking more often on an hours-to-days job).
- **Patience (checkpoints)** — an integer field in both groups, and it's the *same rule* for both: stop automatically once N checkpoints in a row pass with no improvement in the model-selection metric (Full-brain Dice for MONAI — higher is better; `test_loss` for Cellpose-SAM — lower is better; the plugin handles the direction per metric automatically). `0` disables early stopping entirely. This is enforced by the plugin itself, reading each checkpoint as it lands in the log — `train.py`'s own built-in `--patience` flag is always overridden to an effectively-infinite value so it can't quietly stop training before the GUI's check does; there's exactly one early-stopping mechanism, not two that happen to look the same in the UI.
- **Reopening napari mid-training** — the plugin remembers the running job (including the patience setting) and automatically reconnects to it, so you don't lose visibility into a training run just because you closed and reopened napari. You'll see “Resumed monitoring PID ...” instead of an empty status.
- **Reopening napari after the job already finished** — if the training process is no longer running by the time you reopen napari (e.g. it ran to completion, or crashed, while napari was closed), the status line reports that immediately: “Training (PID ...) finished while

napari was closed. Best ... at epoch ...”. For Cellpose-SAM this is also the point where the recommended-checkpoint pointer (see below) gets written, if it wasn’t already. Either way you don’t need to have napari open at the exact moment a run finishes — the next time you open it, it tells you what happened.

- **Email notification** — see [Email notification \(optional\)](#) above. Unlike the previous two bullets, this one doesn’t depend on ever reopening napari at all — the email arrives on its own schedule regardless.
 - **Stop Training** — kills the training process and everything it spawned (the `conda run` wrapper spawns a child python process, and both are terminated together). Early stopping uses this same kill mechanism internally.
 - **Which checkpoint to use afterwards** — MONAI’s `train.py` already tracks and saves its own best checkpoint as `best_model_fullstack.pth`, so nothing extra is needed there. `train_xyz.py` (Cellpose-SAM) has no such tracking — it only saves periodic epoch checkpoints — so whenever the plugin observes a Cellpose-SAM run has stopped (finished on its own, early-stopped, or discovered already-finished the next time you reopen napari — see above), it writes a small pointer file, `<model_name>_best_recommended.txt`, into the run’s `models/` folder next to the checkpoints. It’s a one-line text file naming the best-scoring checkpoint (by `test_loss`), e.g. `cpsam_microglia_xyz_epoch_0150` — not a copy of the (often 100s-of-MB) checkpoint itself, and not an OS symlink either (those need elevated privileges/Developer Mode on Windows), so it works the same way on every platform with no special permissions. The GUI’s status line also reports the best epoch directly once the run stops.
 - **If a script isn’t found** — `prepare_data.py/train.py/train_xyz.py` ship with the plugin (bundled under `napari_zf_microglia_ai/training_scripts/`, installed as package data), so this shouldn’t normally happen. If you want to point at a locally modified copy instead, override the path via the `monai_prepare_script_path/monai_train_script_path/cellpose_train_script_path` keys in `~/config/napari-zf-microglia-ai/config.json`.
-

9. Output files and folder structure

All files saved by the plugin go into a dedicated folder named after your original input file:

```

/path/to/your/data/
├─ NT54_ch1.ims          ← original input file
├─ NT54_ch1/             ← output folder (created
automatically)
    ├─ NT54_ch1_brain_mask.tif ← binary brain mask
    (0/255, uint8)
    ├─ NT54_ch1_brain_only_NoBG.tif ← brain only, background
removed globally
    ├─ NT54_ch1_labels.tif    ← cell labels (int32)
    └─ NT54_ch1_statistics.csv ← per-label statistics

```

The folder is created the first time a file is saved. If no input file has been opened (e.g. you loaded a layer directly in napari), files are saved in the current working directory.

Brain-only suffixes depending on background mode:

Mode	Suffix
Off	<i>(none)</i>
1 — Exterior Removed	_ExtRm
2 — No Background	_NoBG
3 — Random Fill	_RndFill

10. Statistics CSV — all columns explained

The CSV produced by Generate Statistics has one row per label, with up to 51 columns. 46 are always present; the remaining columns appear only when the corresponding optional feature is enabled. This section explains what each column *means*; for the algorithm/formula/library behind each one, see the separate [STATISTICS GUIDE.md](#).

Identification

Column	Type	Description
label	integer	Label number matching the napari Labels layer (1, 2, 3, ...)

Volume

Column	Type	Description
volume_vox	integer	Number of voxels belonging to this label
volume_um3	float (μm^3)	Physical volume in cubic micrometres. Computed as $\text{volume_vox} \times \text{Z_size} \times \text{Y_size} \times \text{X_size}$. A typical zebrafish microglia is 1,000–10,000 μm^3 .

Position (centroid)

The centroid is the 3D centre of mass of the label — the average position of all its voxels.

Column	Type	Description
centroid_z_vox	float	Z position in voxel units
centroid_y_vox	float	Y position in voxel units
centroid_x_vox	float	X position in voxel units
centroid_z_um	float (μm)	Z position in micrometres
centroid_y_um	float (μm)	Y position in micrometres
centroid_x_um	float (μm)	X position in micrometres

Bounding box

The smallest rectangular box (aligned with the axes) that completely contains the label.

Column	Type	Description
bbox_dz_um	float (μm)	Height of the bounding box in Z (depth of the cell in the axial direction)
bbox_dy_um	float (μm)	Height of the bounding box in Y
bbox_dx_um	float (μm)	Width of the bounding box in X

Size and shape

Column	Type	Description
eq_diam_um	float (μm)	Equivalent sphere diameter — the diameter of a perfect sphere with the same volume as this label. Formula: $(6V/\pi)^{1/3}$. Useful as a single “size” number regardless of shape.
axis1_um	float (μm)	Longest principal axis — the maximum extent of the label along its longest geometric direction. Derived from the inertia tensor eigenvectors.
axis2_um	float (μm)	Middle principal axis — approximated as the average of axis1 and axis3.
axis3_um	float (μm)	Shortest principal axis — the minimum extent perpendicular to the longest axis.
elongation	float	Elongation ratio = $\text{axis1} / \text{axis3}$. A perfect sphere = 1.0. A cigar-shaped cell = 3.0 or more. The higher the number, the more stretched out the cell is.
principal_axis_dir	string	The anatomical direction of the longest axis: "Z" (axial), "Y" (coronal), or "X" (sagittal). Tells you which direction the cell is elongated in.

Surface and compactness

Column	Type	Description
solidity	float (0–1)	Solidity = $\text{volume} / \text{convex_hull_volume}$. The convex hull is the smallest convex shape enclosing the label (like shrink-wrap). A

Column	Type	Description
extent	float (0–1)	solid, convex cell = 1.0. A lobulated or branchy cell with lots of indentations < 1.0. Typical range for microglia: 0.5–0.9. Extent = volume / bounding_box_volume. How much of the bounding box is actually filled. A cube = 1.0. A sphere ≈ 0.52. Highly branched cells = much lower.
surface_area_um2	float (μm^2)	Surface area in square micrometres, computed using marching cubes — a 3D mesh is generated from the label boundary and the triangle areas summed. A cell with long thin branches has a much larger surface area than a smooth sphere of the same volume.
sphericity	float (0–1)	Sphericity = $\pi^{1/3} \times (6V)^{2/3} / A$ where V = volume and A = surface area. A perfect sphere = 1.0. Anything less than 1.0 is less spherical. Microglia: typically 0.3–0.8 depending on branch complexity.
surface_to_volume_ratio	float (μm^{-1})	Surface-to-volume ratio = surface_area / volume. Higher values indicate more complex, surface-rich morphology relative to cell size. Branches and protrusions increase this dramatically.

Skeleton (branching structure)

These columns require the `skan` package. If `skan` is not installed, they will be 0.

The algorithm skeletonizes the label (reduces it to a 1-voxel-wide skeleton) and analyses the resulting graph of branches.

Column	Type	Description
<code>n_branches</code>	integer	Number of skeleton branches. A sphere = 1 branch. A microglia with 4 protrusions = roughly 4–8 branches depending on how they connect.
<code>n_endpoints</code>	integer	Number of free-end branch tips (branches that don't loop back). Corresponds roughly to the number of protrusion tips.
<code>mean_branch_len_um</code>	float (μm)	Average path length of all skeleton branches in micrometres.
<code>max_branch_len_um</code>	float (μm)	Length of the longest individual branch — an indicator of maximum protrusion reach.
<code>branch_tortuosity</code>	float (≥ 1)	Average ratio of path length to straight-line distance per branch. A value of 1.0 = perfectly straight branches. Higher values = winding, curved protrusions.
<code>branch_density</code>	float (per $10^6 \mu\text{m}^3$)	Number of branches per million cubic micrometres of cell volume. Allows fair comparison between cells of different sizes.
<code>endpoint_density</code>	float (per $10^6 \mu\text{m}^3$)	Number of branch tips per million cubic micrometres. A proxy for protrusion count normalised by cell volume.
<code>process_complexity</code>	float	Combined measure of branching complexity: $\text{n_branches} \times \text{mean_branch_len} /$

Column	Type	Description
		eq_diam. High values = many long branches relative to cell diameter.

Morphotype classification

Column	Type	Description
morphotype	string	Automatic shape classification based on elongation, sphericity, solidity, branch count, and surface-to-volume ratio. Categories: Rod-shaped (elongated, few branches), Amoeboid (round, compact, few branches), Ramified (many long branches, low sphericity), Intermediate-ramified (moderate branching), Intermediate (doesn't fit the above).

Spatial relationships

These columns use all cell centroids together to compute neighbourhood statistics.

Column	Type	Description
nearest_neighbor_dist_um	float (μm)	Distance to the closest other cell centroid. Small values = cells are tightly packed; large values = isolated cells.
nearest_neighbor_ratio	float	Clark-Evans 3D index for this cell: the ratio of its nearest-neighbour distance to the expected distance if cells were randomly distributed at the same density. Values < 1 =

Column	Type	Description
		clustering; > 1 = regularity/dispersion.
local_density_100um	float (cells/ 10 ⁶ μm ³)	Number of other cells within a 100 μm radius sphere, normalised by sphere volume. A measure of local neighbourhood crowding.
depth_normalized	float (0–1)	Z position normalised to the full depth range of all cells: 0 = shallowest cell, 1 = deepest. Useful for comparing dorsal vs. ventral distribution across samples.

Intensity statistics (*optional — requires Image layer selection*)

Column	Type	Description
mean_intensity	float	Mean pixel intensity inside the label mask. Reflects overall fluorescence brightness of the cell.
integrated_intensity	float	Sum of all pixel values inside the label (mean × voxel count). Proportional to total fluorescent material in the cell regardless of size.
intensity_cv	float (0–∞)	Coefficient of variation of pixel intensities = std / mean. 0 = perfectly uniform. High values = heterogeneous staining, possibly indicating internal structure or imaging artefacts.

Brain region assignment (*optional — requires Shapes layer with boundary lines*)

Column	Type	Description
brain_region	string	Name of the anatomical region this cell belongs to (as defined by the boundary lines and region names you provided).
region_boundary_dist_um	float (μm)	Distance from this cell's centroid to the nearest region boundary line, in micrometres. Cells near boundaries may have mixed characteristics.

Description

Column	Type	Description
description	string	A plain-language sentence summarising the cell's shape, generated by the selected description backend. Example (rule-based): <i>"Label 3: Elongated along Y-axis (2.8:1), volume 4,521 μm³, centroid Z=87.3 Y=142.1 X=203.5 μm. Lobulated/irregular surface, sphericity 0.41, solidity 0.72. Morphotype: Intermediate-ramified. 6 branches, 4 endpoints (mean 8.3 μm), tortuosity 1.4."</i>

11. Setting up description backends

Rule-based (offline) — no setup needed

The default. Descriptions are generated using built-in templates based on the numeric values. No internet connection, no API key, no external software. Always available.

Ollama (local, free)

Ollama runs a large language model locally on your machine. No data is sent to external servers, and there is no ongoing cost after the initial download.

Step 1 — Install Ollama

Go to <https://ollama.com/download> and download the installer for your operating system. Run it.

- On Linux: `curl -fsSL https://ollama.com/install.sh | sh`
- On Mac: Download the .dmg and drag to Applications.
- On Windows: Download the .exe installer.

Step 2 — Download a model

Open a terminal and run:

```
ollama pull llama3
```

This downloads the Llama 3 model (~4.7 GB). You only need to do this once. Other models you can use:

```
ollama pull mistral      # ~4 GB, fast
ollama pull phi3         # ~2 GB, smaller and faster
ollama pull llama3:70b   # ~40 GB, highest quality – needs 64 GB+
                        RAM
```

Step 3 — Verify Ollama is running

Ollama starts automatically in the background after installation. You can confirm it is running:

```
ollama list  # should show your downloaded models
```

Step 4 — Configure in the plugin

In Tab 3 — Statistics:

1. Select **Ollama (local, free)** from the Description dropdown.
2. **Endpoint:** leave as `http://localhost:11434` (default). Only change this if Ollama runs on a different machine on your network.
3. **Model:** type the model name you downloaded, e.g. `llama3`.
4. Click **Generate Statistics**.

If you get an `[Ollama error: ...]` in the CSV description column, check that Ollama is running (`ollama list`) and that the model name matches exactly what you downloaded.

OpenAI API (paid)

OpenAI's GPT models run on OpenAI's servers. You pay per token processed. For statistics descriptions (short prompts, short responses), the cost is very low — roughly \$0.001–0.01 per 100 cells with `gpt-4o-mini`.

Step 1 — Create an OpenAI account

Go to <https://platform.openai.com> and sign up. You will need to provide a credit card for billing.

Step 2 — Generate an API key

1. Log in to <https://platform.openai.com>.
2. Click your profile icon (top right) → **API keys**.
3. Click + **Create new secret key**.
4. Give it a name (e.g. “`napari-zf-microglia-ai`”).
5. Copy the key immediately — it starts with `sk-` and you can only see it once.

Step 3 — Configure in the plugin

In Tab 3 — Statistics:

1. Select **OpenAI API (paid)** from the Description dropdown.
2. **API Key:** paste your `sk-...` key. It is stored only in memory — not saved to disk.
3. **Model:** `gpt-4o-mini` (recommended — low cost, good quality). Other options:
 - `gpt-4o` — highest quality, higher cost
 - `gpt-3.5-turbo` — fastest, cheapest, lower quality
4. **Base URL:** leave blank unless you use an OpenAI-compatible proxy.

5. Click **Generate Statistics**.

The API key is **not saved to disk** for security. You must paste it again each time you open napari.

Claude API (paid)

Anthropic's Claude models. Similar pricing model to OpenAI. Claude Haiku is very fast and inexpensive.

Step 1 — Create an Anthropic account

Go to <https://console.anthropic.com> and sign up with a credit card.

Step 2 — Generate an API key

1. Log in to <https://console.anthropic.com>.
2. Click **API Keys** in the left sidebar.
3. Click + **Create Key**.
4. Give it a name and copy the key (starts with sk-ant-).

Step 3 — Configure in the plugin

In **Tab 3 — Statistics**:

1. Select **Claude API (paid)** from the Description dropdown.
 2. **API Key:** paste your sk-ant-... key.
 3. **Model:** claude-haiku-4-5-20251001 (recommended — fast and cheap). Other options:
 - claude-sonnet-4-6 — higher quality, moderate cost
 - claude-opus-4-6 — highest quality, highest cost
 4. **Base URL:** leave blank (not used for Claude).
 5. Click **Generate Statistics**.
-

11a. Setting up email notification (Gmail App Password)

The **Email notification** panel in **Tab 4 — AI Tools** (see Section 8) sends one email when a training run stops, even if napari isn't open at the time. It works with any SMTP-over-SSL provider, but Gmail is the easiest and free path — this walks through it end to end. If you'd rather use a different provider (Outlook/Office365, a work email server, etc.), skip to **Using a non-Gmail provider** at the bottom.

Step 1 — Turn on 2-Step Verification (if not already on)

App Passwords only exist for Google accounts with 2-Step Verification enabled — this is a Google requirement, not something the plugin asks for.

1. Go to <https://myaccount.google.com/security>.
2. Under “How you sign in to Google,” click **2-Step Verification**.
3. Follow the prompts to turn it on (usually a phone number + a code sent by SMS or the Google Authenticator app).

If it’s already on, skip straight to Step 2.

Step 2 — Generate an App Password

1. Go to <https://myaccount.google.com/apppasswords> (you may be asked to sign in again).
2. Under “App name,” type something recognizable, e.g. `napari-zf-microglia-ai`.
3. Click **Create**.
4. Google shows a 16-character password (four groups of four letters, e.g. `abcd efgh ijkl mnop`). Copy it now — this is the only time it’s shown. Spaces don’t matter; you can paste it with or without them.

This is **not your normal Gmail password** and won’t be accepted as one — Google deliberately issues a separate, revocable password for exactly this kind of use (a third-party app sending mail on your behalf). You can revoke it any time from the same App Passwords page without affecting your main account password.

Step 3 — Configure in the plugin

In **Tab 4 — AI Tools**, in the **Email notification (optional)** panel above the MONAI/Cellpose-SAM switch:

1. **Notify email:** the address that should receive the notification — typically your own Gmail address, but it can be any address you want the report sent to.
2. **SMTP server:** leave as `smtp.gmail.com` (the default).
3. **port:** leave as 465 (the default).
4. **SMTP username:** your full Gmail address (e.g. `you@gmail.com`).
5. **SMTP password:** paste the 16-character App Password from Step 2 — *not* your normal Gmail password.
6. Click **Launch Training** as usual. You should get one email the next time that run stops (finishes, crashes, or gets early-stopped).

The SMTP password is **not saved to disk** for security (same policy as the API keys in Section 11) — you’ll need to paste it again each time you open napari and want notifications for a

new run. **Notify email/SMTP server/port/SMTP username** are remembered between sessions, since none of those are secret on their own.

Using a non-Gmail provider

Any SMTP server that supports SSL on a fixed port works the same way — just change **SMTP server/port** to match your provider and use whatever credentials it issues (an app-specific password if the provider offers one, same reasoning as Gmail’s). A few examples:

Provider	SMTP server	Port
Gmail	smtp.gmail.com	465
Outlook / Office 365 (personal)	smtp.office365.com	587 (<i>see note below</i>)
Yahoo Mail	smtp.mail.yahoo.com	465

Note: the plugin’s supervisor script always connects via SMTP_SSL (implicit TLS from the first byte, no STARTTLS handshake) — this matches Gmail and Yahoo’s port-465 behavior. Providers that only offer STARTTLS on port 587 (like Office 365) are not currently supported without a small code change; Gmail is the tested, recommended path.

12. Full workflow: from raw stack to labelled cells

Step 1 — Open your file

- 1. Open the plugin (Plugins → Main Panel (ZF-Microglia-AI)) in napari.
- 2. Click **Open TIF / IMS file** and select your confocal stack.
- 3. All channels appear as layers.
- 4. **Click the microglia channel** (usually ch1, green) in the Layers panel.

Step 2 — Run skin removal

Set these values in Tab 1:

Setting	Value
MONAI Threshold	0.25
Erosion	0 (default)
Background	Option 1 if you plan to use Cellpose-SAM in Step 3, Option 2 if you plan to use the Pixel Classifier
BG Threshold	1.40

Click **Run Skin-Remover** and wait.

What you should see: A brain_only layer. With Option 2, microglia appear as bright isolated blobs on a black background, with clear space between cells (needed for the Pixel Classifier). With Option 1, the brain interior is left intact — only the tissue outside the brain is removed (needed for Cellpose-SAM).

If blobs look hollow or have large halos (Option 2): Lower BG Threshold (e.g. 0.40).

If too much dim signal remains between cells (Option 2): Raise BG Threshold (e.g. 0.80).

Step 3 — Create labels

Click the brain_only layer Tab 1 just produced (_ExtRm or _NoBG, matching your choice above) in the Layers panel, then switch to the **Create Labels** tab. It automatically shows the matching section — see [Section 6a](#) for the full logic.

Option A — Pixel Classifier (active layer ends in _NoBG)

Set these values:

Setting	Value
Smooth σ XY	1.5
Smooth σ Z	3.0
Min overlap	10% (default)
Min volume	7500 (default)

Click **Create Labels**.

What you should see: A labels layer where each cell is a different colour. The console prints how many were found.

Tuning: - Too many tiny fragments → increase Min volume or increase both σ values - Two cells merged together → try Split Label (see below)
- Cells cut across slices → decrease Min overlap or increase σ Z

Option B — Cellpose-SAM Segmentation (active layer ends in _ExtRm)

1. Browse to your Cellpose-SAM checkpoint (Section 3) if not already set.
2. Leave the defaults (Cellprob -2.5, Flow 0.4, Safe-merge max gap 2, Safe-merge min contact 10, Large-contact merge 20) unless you know you need to adjust them — see [Section 6c](#) for what each one does.
3. Click **Run Cellpose-SAM Segmentation** and wait — this can take hours for a full-size fish. Progress is shown in the status bar; napari stays usable while it runs.

What you should see: A labels layer where each cell is a different colour, exactly as with the Pixel Classifier.

Step 4 — Review and edit labels in napari

- Toggle the labels layer on/off to compare with the original
 - Hover over cells to see their label number
 - Zoom through Z slices to verify cells are correctly separated
-

Step 5 — Split merged cells (if needed)

If two cells were labelled as one because they touch:

1. Hover over the merged blob and note its label number (shown in the napari status bar at the bottom).
2. In Tab 2, under **Split Label**:
 - **Target label:** enter the label number (or click the blob and click **Use selected**).
 - **Split into:** 2 (or however many cells are merged).
 - **Smooth σ :** 1.0 (default).
 - **Min distance:** 5 (default).
3. Click **Split Label**.

4. The two (or more) cells are separated at their thinnest connection point.
-

Step 6 — Sort labels (optional)

Click **Resort Labels** to renumber cells by size or position. This is helpful for consistent reporting:

- By **Size** (largest = label 1) — most common
 - By **Centroid Z/Y/X** — for atlas alignment
-

Step 7 — Save labels

Click **Save Labels**. A file dialog opens pre-filled with the output folder. Accept or change the name and click Save.

Step 8 — Generate statistics

1. Click the **Statistics** tab (Tab 3).
 2. Make sure the Labels layer is selected in napari.
 3. Choose your description backend.
 4. *(Optional)* Select a fluorescence channel under **Intensity statistics** to add mean/integrated/CV columns.
 5. *(Optional — see Step 8a below)* Draw region boundary lines in a Shapes layer, then select it under **Brain regions** and enter the region names (e.g. Optic tectum, Hindbrain).
 6. Click **Generate Statistics**.
 7. The CSV is saved automatically to the output folder.
-

Step 8a — Assign cells to brain regions (optional)

This lets you label each cell as belonging to the **optic tectum**, the **hindbrain**, or any other anatomical region you define by drawing a dividing line across the image.

Orientation reminder: The fish lies along the **X axis** — head at $X = 0$, tail at $X = \text{max}$. Y runs top to bottom ($0 \rightarrow 2048$). The optic tectum / hindbrain boundary therefore appears as a roughly vertical curve when you look at the XY plane — it runs from the top of the brain

(small Y) to the bottom (large Y), at some X position. Everything to the **left** of the curve (smaller X) is the optic tectum; everything to the **right** (larger X) is the hindbrain.

Step-by-step:

1. **Scroll to a representative Z slice** where the optic tectum / hindbrain boundary is most clearly visible. In zebrafish 4dpf, this boundary is typically a recognisable change in cell density roughly at the mid-point of the anterior–posterior (X) axis.
2. **Add a Shapes layer:** In napari, click the + icon in the toolbar and choose **Shapes**, or go to **Layers → Add shapes layer**.
3. **Select the path tool:** In the shapes toolbar (appears when the Shapes layer is active), click the **path** icon (a polyline with multiple vertices). Do **not** use the straight line tool — the optic tectum / hindbrain boundary is curved.
4. **Draw the boundary curve from top to bottom:** Start clicking at the top of the brain (small Y, $Y \approx 0$ side) and work downward to the bottom of the brain (large Y). Follow the curved anatomical boundary as you click. Double-click on the last point to finish. You typically need 4–10 click points.
 - The optic tectum (anterior, smaller X) will be on the **left** of your drawn path.
 - The hindbrain (posterior, larger X) will be on the **right**.

Tip: zoom in on the XY view where the boundary is clearest. If unsure of the exact position, trace the curve slightly anterior (more to the left). You can always select the path layer, press Delete to remove it, and redraw.

5. *(For three or more regions)* Draw one additional path per boundary, each running top to bottom.
6. **Switch to Tab 3 — Statistics** in the plugin.
7. Under **Brain regions**:
 - **Boundary lines:** select your Shapes layer from the dropdown.
 - **Region names:** type the names separated by commas, anterior to posterior:

Optic tectum, Hindbrain

For three regions, e.g.: Forebrain, Optic tectum, Hindbrain

8. Click **Generate Statistics**.

Result: The CSV will include two extra columns:

Column	Description
brain_region	Name of the region each cell belongs to
region_boundary_dist_um	Distance in μm from the cell to the nearest boundary line

You can then filter the CSV in Excel or Python by `brain_region` to compare microglia density, morphology, or intensity between the optic tectum and the hindbrain.

13. Reinstalling after an update

```
pip uninstall napari-zf-microglia-ai -y
pip install git+https://github.com/CTichy/ZF-Microglia-AI.git
```

Then **fully close and reopen napari**. If napari is running when you reinstall, it uses the old version until restarted.

Your model path and settings are preserved across reinstalls. The config is stored in `~/.config/napari-zf-microglia-ai/config.json`.

14. Troubleshooting

conda env create -f environment.yml fails on Windows with Didn't find wheel for cucim-cu12

Fixed in the current `environment.yml` (`cucim-cu12` is now Linux-only there — it has no Windows wheels at all, since it's a Linux/WSL2-only RAPIDS package, and isn't something a different pip flag or index fixes on native Windows). It only accelerates Tab 3 statistics, which fall back to CPU cleanly without it; Tab 1 and Tab 2 are unaffected.

If you still hit this error:

- You're on an older clone — `git pull` in the repo folder, then retry.

- If the environment partially exists from the earlier failed attempt, remove it first: `conda env remove -n zf-microglia-ai`, then `conda env create -f environment.yml` again.
-

The plugin does not appear in Plugins menu

- Make sure napari is fully closed and reopened after installation.
 - Verify installation: `pip show napari-zf-microglia-ai`
-

“No model selected” after reinstalling

- Click [...] and browse to your .pth file.
 - Config path: `~/.config/napari-zf-microglia-ai/config.json`
-

Tab 4 (AI Tools) is showing a red or amber warning banner

This is expected, not an error — see Section 8. The tab is always available regardless of GPU; the banner just tells you what to expect. No CUDA GPU means CPU fallback (days-months for a full training run instead of hours); a GPU under 8GB may still work with a lower `batch_size` (try 2, or even 1) before assuming something else is wrong.

“Launch Training” errors that a script wasn’t found

`prepare_data.py/train.py/train_xyz.py` ship with the plugin (bundled under `napari_zf_microglia_ai/training_scripts/`, installed as package data), so this shouldn’t normally happen. If you want to point at a locally modified copy instead, override the path in `~/.config/napari-zf-microglia-ai/config.json` — keys `monai_prepare_script_path`, `monai_train_script_path`, `cellpose_train_script_path`.

Processing runs on CPU (very slow)

NVIDIA GPU: Check that PyTorch sees CUDA:

```
python -c "import torch; print(torch.cuda.is_available())"
```

Should print `True`. If `False`, reinstall PyTorch with CUDA support from <https://pytorch.org>.

Apple Silicon: Check MPS:

```
python -c "import torch; print(torch.backends.mps.is_available())"
```

Should print `True` on M1/M2/M3.

Statistics are slow (CPU only, no GPU batch)

Linux: Install CuPy and cuCIM for GPU-accelerated regionprops:

```
pip install cupy-cuda12x cucim-cu12
```

Replace `cuda12x/cu12` with your actual CUDA version if different (e.g. `cuda11x` for CUDA 11). After installing, reopen napari — the console will print `regionprops: cuCIM GPU` when statistics are computed.

Windows: `cucim-cu12` has no native Windows build (RAPIDS/cuCIM is Linux/WSL2-only) — this is expected and not fixable with a different pip command on native Windows. Tab 3 statistics run on a CPU-threaded path instead; Tab 1 inference and Tab 2 GPU labelling are unaffected, since those only need `cupy-cuda12x`, which does support Windows. If GPU-accelerated statistics specifically matter to you, run the plugin inside WSL2 (Windows Subsystem for Linux) instead, where the Linux install path applies.

brain_only layer looks mostly empty (all black)

BG Threshold is too high — lower it (e.g. from 1.40 toward 0.50-0.60).

Create Labels finds 0 or too few objects

- Lower **Min volume** (try 5000).
 - Increase σ_{XY} and σ_Z slightly.
 - Make sure you selected the `brain_only` layer (not the raw channel) before clicking Create Labels.
-

Create Labels finds hundreds of tiny fragments

- Increase **Min volume** to 10000.
 - Ensure Option 2 with sufficient BG Threshold was used — the brain_only layer must have clean gaps between cells.
-

Two cells appear as one label (merged)

- Use **Split Label** (Section 6 above) to separate them at the thinnest neck.
 - Or decrease σ XY and rerun Create Labels.
-

Split Label error: “Only N sub-volume(s) found”

The blob doesn’t have a clear separation into the requested number of parts.

- Reduce **Smooth σ** (the distance field is over-smoothed and the saddle disappears).
 - Reduce **Min distance** (the two centres are being rejected as too close).
 - Check the blob is genuinely two distinct cells — zoom in and inspect it slice by slice.
-

Ollama description shows [Ollama error: ...]

- Verify Ollama is running: open a terminal and run `ollama list`.
 - If not running, start it: `ollama serve`
 - Check the model name matches exactly: `ollama list` shows available models.
 - Default endpoint `http://localhost:11434` — change only if Ollama is on a different machine.
-

OpenAI/Claude API returns an error

- The API key must be pasted fresh each session — it is not saved to disk.
- Check your account has billing set up and enough credit.

- The model name must match exactly (e.g. gpt-4o-mini, claude-haiku-4-5-20251001).

“Run Cellpose-SAM Segmentation” errors with No module named 'cellpose'

Install it in your environment: `pip install cellpose`. Already included in `environment.yml/environment-mac.yml` for fresh installs — if you set up your environment before this feature was added, run `conda env update --name zf-microglia-ai -f environment.yml --prune` (or `environment-mac.yml`) and reinstall.

Neither Pixel Classifier nor Cellpose-SAM section shows up in Tab 2

The active layer’s name must end in `_ExtRm` (Cellpose-SAM) or `_NoBG` (Pixel Classifier) — reselect the correct Tab 1 output layer in the Layers panel. See [Section 6a](#).

Quick Reference Card

Tab 1 — Skin Remover

Control	Recommended	What it does
MONAI Threshold	0.25	AI confidence cutoff
Erosion	0	Strips voxels from mask edge
Background	Option 2	Removes background globally (best for labels)
BG Threshold	1.40	Fine-tunes background removal level
Verify MONAI Threshold / Erosion (GT Sweep)	5x5 grid	Confirms current values against a hand-corrected GT brain mask — MONAI runs once, rest is cheap — best point auto-

Control	Recommended	What it does
		applied to the sliders and saved

Tab 2 — Create Labels

Shown automatically based on active layer suffix — `_ExtRm` → Cellpose-SAM, `_NoBG` → Pixel Classifier (see [6a](#)).

Pixel Classifier

Control	Recommended	What it does
Smooth σ XY	1.5	Contour softness within each slice
Smooth σ Z	3.0	Cross-slice blob connectivity
Min overlap	10%	Overlap needed to link blobs across slices
Min volume	7500	Minimum voxels to keep a 3D object
Verify BG Threshold / Erosion (GT Sweep)	5x5 grid	Confirms current BG Threshold/Erosion against real GT IoU — CPU-OK, doesn't survive closing napari — best point auto-applied to the sliders and saved

Cellpose-SAM Segmentation

Control	Recommended	What it does
Cellprob threshold	-2.5	Confidence cutoff for foreground vs. background
Flow threshold	0.4	Rejects self-inconsistent flow predictions
Safe-merge max gap	2 vox	Max gap allowed when merging fragments
Safe-merge min contact	10 vox	Min touching surface required to merge
	10230 vox	

Control	Recommended	What it does
Safe-merge GT-min volume		Smallest volume trusted as already a whole cell — recalibrated from real GT by the sweep below, not usually set by hand
Large-contact merge	20 vox	Second merge pass for thick-junction splits
Verify Cellprob / Large-contact (GT Sweep)	5x5 grid	Confirms against whole-fish GT — Cellprob needs re-inference (GPU-preferred), Large-contact is cheap — best point + measured GT-min auto-applied to the sliders and saved
Build GT-Correction Package	—	Zips Krendl output + stats CSV + creation guide for external manual correction

Both methods (once labels exist)

Control	Recommended	What it does
Split σ	1.0	Smoothness for watershed split
Min distance	5	Peak separation for split detection

Tab 3 — Statistics

Control	Options	What it does
Description	Rule-based / Ollama / OpenAI / Claude	Engine for the description column
Image layer	Any Image layer / None	Adds intensity statistics (mean, integrated, CV)
Boundary lines	Any Shapes layer / None	Assigns cells to named brain regions
Region names	Comma-separated text	Names for each region (N lines → N+1 names)
	—	

Control	Options	What it does
Generate Statistics		Computes up to 45 metrics per label, saves CSV
Score Against GT	any 2 Labels layers	Whole-fish Hungarian-matched TP/FP/FN/Score/MeanIoU/MeanDice — synchronous, no GPU needed

Tab 4 — AI Tools

Always shown — a banner at the top warns if your GPU is missing or under the recommended 8GB (see Section 8), but doesn't block anything. Switch below it picks MONAI or Cellpose-SAM training.

Control	Default	What it does
Notify email	<i>(blank = off)</i>	Shared — one email when a run stops (finish/crash/early-stop), even if napari never reopens
SMTP server / port / username / password	smtp.gmail.com / 465 / — / —	Shared — password never persisted, re-enter each session
n_val / n_test	5 / 5	(MONAI) fish held out for val/test in Prepare Training Data
epochs	1500	(MONAI) training length
n_epochs	200	(Cellpose-SAM) training length
branch_weight	0	(Cellpose-SAM) 0 = standard loss; >0 weights thin/branch pixels more heavily
branch_radius	3 px	(Cellpose-SAM) erosion-survival distance threshold for the branch-weighted loss — measurable from real GT via Calibrate branch_radius below
Calibrate branch_radius (from GT)	—	(Cellpose-SAM) measures real branch thickness from a GT labels volume (3D skeleton + distance transform) — recommendation auto-applied to branch_radius and saved

Control	Default	What it does
pretrained	Tab 2's checkpoint	(Cellpose-SAM) starting point — “continue training” by default
Extract XXYZ Patches	crop_size=512	(Cellpose-SAM) current production crop method — 3 orientations, cleans truncated labels by default
Patience (checkpoints)	5	Both — stop after N checkpoints with no improvement (Dice/test_loss); 0 disables
Launch Training	—	Starts a detached process that survives closing napari; GUI reconnects automatically next time
(on stop, Cellpose-SAM only)	—	Writes <model_name>_best_recommended.txt in models/ — a pointer, not a copy, to the best-test_loss checkpoint
Verify Best Epoch (GT Sweep)	5 cells, ±2 checkpoints	(Cellpose-SAM) confirms the recommendation against real GT IoU/Dice, not just test_loss — doesn't survive closing napari — if the sweep disagrees, rewrites the pointer to the confirmed epoch and loads it as Tab 2's active model
Stop Training	—	Kills the training process and its children

Plugin developed at FH Technikum Wien — Artificial Intelligence & Data Science Contact: carlos.tichy@gmail.com